

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ИНДИВИДУАЛЬНОЕ ДОМАШНЕЕ ЗАДАНИЕ
по дисциплине «Разработка ПО информационных систем»
Тема: Платформа для запуска пользовательских программ-ботов
для
бесконечных крестиков-ноликов

Студенты гр. 3344

Кузнецов Р.А.

Гусева Е.А.

Мурдасов М.К.

Преподаватель

Заславский М.М.

Санкт-Петербург

2026

ЗАДАНИЕ

Студенты:

Кузнецов Р.А.

Гусева Е.А.

Мурдасов М.К.

Группа: 3344

Тема проекта: Разработка приложения для хранения, запуска и анализа пользовательских программ-ботов для игры в бесконечные крестики-нолики.

Исходные данные:

Необходимо реализовать веб-приложение с хранением данных в MongoDB, поддержкой загрузки ботов, запуска матчей, просмотра истории партии, логов, статистики, импорта и экспорта данных. В качестве дополнительного результата требуется сравнить нереляционную модель данных с аналогичной SQL-моделью.

Содержание пояснительной записки:

«Содержание»

«Введение»

«Актуальность»

«Постановка задачи»

«Предлагаемое решение»

«Качественные требования»

«Сценарии использования»

«Модель данных»

«Сравнение NoSQL и SQL моделей»

«Описание разработанного приложения»

«Вывод»

«Приложение»

«Список использованных источников»

Предполагаемый объем пояснительной записки:

Не менее 10 страниц.

Дата выдачи задания: _____.

Дата сдачи: _____.

Дата защиты: _____.

Студенты гр. 3344

Кузнецов Р.А.

Гусева Е.А.

Мурдасов М.К.

Преподаватель

Заславский М.М.

АННОТАЦИЯ

В рамках индивидуального домашнего задания разработан прототип веб-платформы для запуска пользовательских программ-ботов в игре «бесконечные крестики-нолики». Приложение позволяет регистрировать пользователей, загружать ботов, создавать матчи, просматривать ход партии, анализировать логи и статистику, а также выполнять импорт и экспорт данных. Основной системой хранения выбрана MongoDB, так как данные матчей и событий имеют полуструктурированный характер и естественно описываются документами. В работе приведены сценарии использования, нереляционная и реляционная модели данных, оценка удельного объема хранения и сравнение моделей по числу запросов. Разработанное приложение состоит из frontend-сервиса, backend API, контейнера MongoDB и изолированного сервиса запуска ботов. Найти исходный код и всю дополнительную информацию можно по ссылке: <https://github.com/moevm/nsq1h26-xo>

SUMMARY

This report describes a prototype web platform for running user-created bots for infinite tic-tac-toe. The application supports user accounts, bot uploads, match creation, replay viewing, logs, statistics, report building, import and export. MongoDB is used as the primary storage because match events and logs are semi-structured and can be stored naturally as documents. The report contains use cases, UI mockups, NoSQL and SQL data models, storage size estimation, query analysis, application architecture and deployment instructions. The implemented system consists of a React frontend, FastAPI backend, MongoDB database and a separate isolated bot runner service. Source code and all the additional information can be found at: <https://github.com/moevm/nsql1h26-xo>

СОДЕРЖАНИЕ

Введение	7
1. Актуальность решаемой проблемы	7
2. Постановка задачи	8
3. Предлагаемое решение	8
4. Качественные требования к решению	9
5. Сценарии использования	10
5.1. Макет UI	10
5.2. Роли пользователей	10
5.3. Основные сценарии	11
5.4. Сценарии импорта данных	12
5.5. Представление данных	12
5.6. Анализ данных	12
5.7. Экспорт данных	13
5.8. Вывод о преобладающих операциях	13
6. Модель данных	13
6.1. Нереляционная модель MongoDB	13
6.2. Примеры документов и запросов MongoDB	15
6.3. Аналог модели для SQL СУБД	17
6.4. Сравнение моделей	18
7. Разработанное приложение	19
7.1. Архитектура и модули	19
7.2. API приложения	20
7.3. Используемые технологии	21
7.4. Снимки экрана приложения	21
7.5. Текущее состояние реализации	21
8. Выводы	22
Приложение А. Документация по сборке и развертыванию	22
Приложение Б. Инструкция пользователя	23
Список использованных источников	23

ВВЕДЕНИЕ

Цель работы - разработать прототип информационной системы для хранения, запуска и анализа пользовательских программ-ботов, соревнующихся в игре «бесконечные крестики-нолики». Система должна обеспечивать хранение файлов решений, запуск матчей между ботами, фиксацию ходов и технических сообщений, просмотр результатов и построение статистики.

В качестве основной СУБД используется MongoDB. Такой выбор связан с тем, что данные предметной области имеют разную структуру: карточка бота, версия файла, матч, поток ходов, логи и отчеты отличаются составом полей, а поток событий может расширяться новыми типами сообщений. Документная модель позволяет хранить такие данные без искусственного дробления на большое количество таблиц.

Результатом работы является веб-приложение Bot Arena Prototype, состоящее из пользовательского интерфейса, серверного API, базы данных и отдельного сервиса безопасного запуска ботов. Исходный код проекта размещается в репозитории: <https://github.com/moevm/nsql1h26-xo>.

1. АКТУАЛЬНОСТЬ РЕШАЕМОЙ ПРОБЛЕМЫ

Соревновательные платформы для программ-ботов используются в обучении программированию, алгоритмам и анализу данных. Они позволяют участникам отправлять свои решения, сравнивать их между собой и получать объективную статистику. Для такой платформы важно не только запускать пользовательский код, но и надежно хранить историю запусков, результаты матчей, ошибки, логи и статистические агрегаты.

Особенность задачи заключается в большом количестве событий, связанных с каждым матчем. В игре с бесконечным полем число ходов заранее неизвестно, координаты могут выходить за стандартные границы, а логические и технические события имеют разную структуру. Поэтому классическая реляционная модель может оказаться менее удобной: для хранения разных типов событий требуется несколько таблиц и дополнительные объединения данных при просмотре матча.

Документная NoSQL-модель дает возможность хранить карточку матча и поток событий в форме, близкой к предметной области. Это упрощает реализацию replay, поиска по логам, статистики и экспорта данных. Поэтому сравнение MongoDB-модели с SQL-аналогом является важной частью работы.

2. ПОСТАНОВКА ЗАДАЧИ

Необходимо разработать веб-приложение, которое предоставляет пользователям платформу для работы с ботами игры «бесконечные крестики-нолики». В системе должны быть роли пользователя, модератора и администратора. Пользователь должен иметь возможность войти в систему, просматривать ботов, создавать матчи и анализировать результаты. Модератор и администратор должны дополнительно управлять ботами, логами, импортом, экспортом и отчетами.

Основные функциональные задачи:

- регистрация и вход пользователей;
- загрузка файлов ботов и хранение версий;
- поиск и просмотр карточек ботов;
- создание матча между двумя ботами с выбранными параметрами;
- сохранение истории ходов, статусов и логов матча;
- просмотр replay-поля и технических сообщений;
- поиск по логам и редактирование заметок;
- построение статистики по ботам и матчам;
- создание пользовательских отчетов;
- импорт и экспорт данных в машиночитаемом формате;
- управление настройками приложения и ролями пользователей.

Дополнительно требуется описать модель данных MongoDB, построить аналогичную SQL-модель, оценить объем хранения и сделать вывод о том, какая модель лучше подходит для проекта.

3. ПРЕДЛАГАЕМОЕ РЕШЕНИЕ

Предлагается реализовать приложение как набор контейнеров Docker Compose. Пользователь работает с веб-интерфейсом React. Интерфейс обращается к backend-сервису на FastAPI, который выполняет авторизацию, обработку запросов, работу с MongoDB и взаимодействие с сервисом запуска ботов. Пользовательский код запускается не внутри backend, а в отдельном контейнере bot-runner с ограничениями по ресурсам.

Данные приложения хранятся в MongoDB. В базе используются коллекции users, bots, bot_versions, bot_stats, matches, match_events, reports, import_export_history, app_settings и log_notes. При старте приложение заполняет базу демонстрационными данными, если коллекция ботов пуста. Это позволяет проверить основные сценарии без ручной подготовки данных.

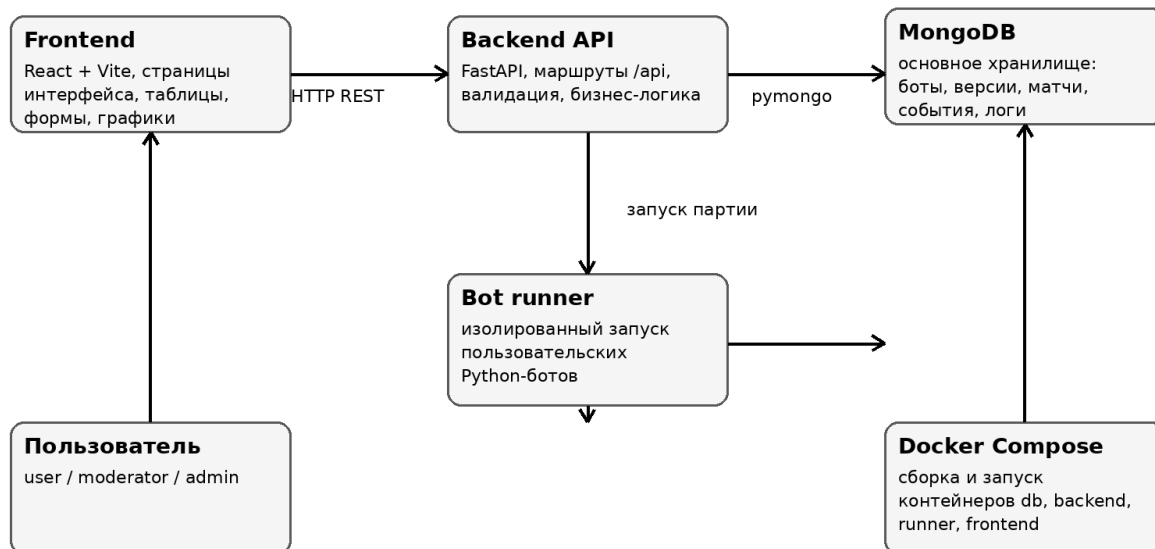


Рисунок 1. Общая архитектура приложения.

Использование отдельного сервиса запуска повышает безопасность и устойчивость системы. В docker-compose для bot-runner задан режим `read_only`, временная файловая система `tmpfs`, запрет повышения привилегий, ограничение числа процессов, памяти и CPU. Это важно, так как загружаемые файлы являются пользовательским кодом.

4. КАЧЕСТВЕННЫЕ ТРЕБОВАНИЯ К РЕШЕНИЮ

Требование	Описание реализации
Удобство использования	Интерфейс разделен на экраны: вход, обзор, боты, матчи, логи, статистика, отчеты, импорт/экспорт и настройки.
Разграничение прав	Роли <code>user</code> , <code>moderator</code> и <code>admin</code> ограничивают доступ к загрузке ботов, редактированию логов, импорту, экспорту и настройкам.
Расширяемость	Новые типы событий матча могут добавляться в <code>match_events</code> без изменения общей структуры базы.
Безопасность запуска	Пользовательские боты выполняются в отдельном контейнере с ограничениями ресурсов.

Надежность хранения	Карточки ботов, версии, матчи, события и отчеты сохраняются в MongoDB.
Аналитика	Статистика и отчеты строятся по матчам, ходам, логам и агрегированным данным ботов.
Импорт и экспорт	Поддерживается полный JSON-дамп данных приложения и история операций.

5. СЦЕНАРИИ ИСПОЛЬЗОВАНИЯ

5.1. Макет UI

Пользовательский интерфейс построен как многостраничное веб-приложение. Основные экраны: вход, регистрация, обзор, список ботов, карточка бота, создание матча, карточка матча, логи, статистика, конструктор отчета, импорт/экспорт и настройки. Общая логика интерфейса показана на рисунке 2.

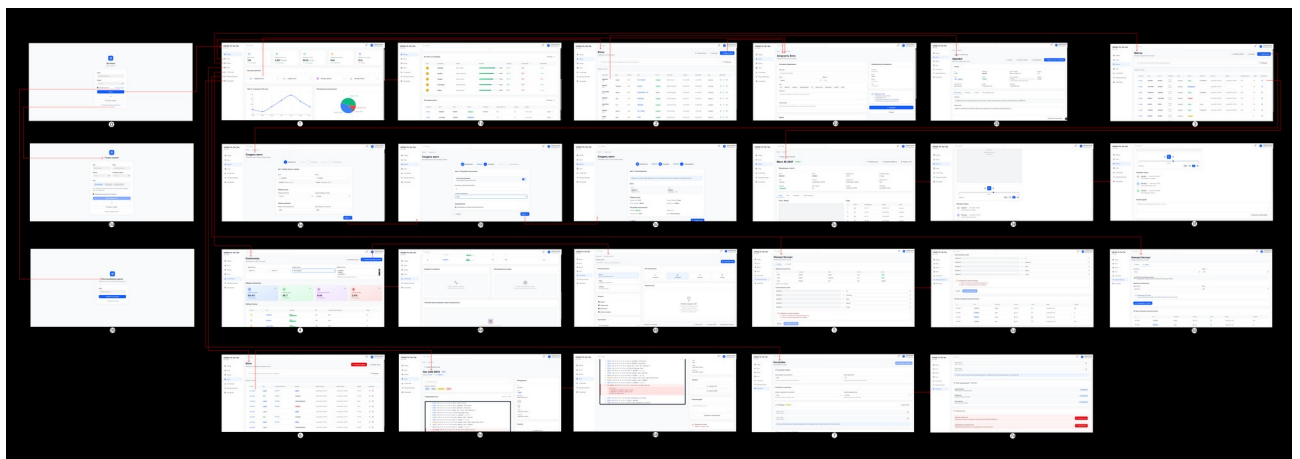


Рисунок 2. Макеты основных экранов приложения.

В прототипе уже реализована навигация по экранам, таблицы данных, формы создания и редактирования сущностей, просмотр карточек, фильтрация и поиск. Последний коммит фронтенда учитывается как часть проекта: в нем расширены экраны статистики, конструктора отчетов, настроек, таблиц и навигации.

5.2. Роли пользователей

Роль	Доступные действия
Пользователь	Вход, регистрация, просмотр ботов и матчей, создание матча, просмотр

	статистики, построение кастомного отчёта по доступным данным.
Модератор	Все действия пользователя, загрузка и редактирование ботов, работа с логами, импорт/экспорт, отчеты и анализ.
Администратор	Все действия модератора, управление пользователями и выдача ролей, изменение настроек приложения.

5.3. Основные сценарии

0. Вход в систему. Пользователь открывает экран «Вход», вводит email и пароль, нажимает «Войти». Система проверяет данные, создает сессию и открывает экран «Главная/Обзор». При неверном пароле отображается сообщение об ошибке.

1. Регистрация. Пользователь открывает экран регистрации, вводит имя, email, пароль и подтверждение пароля, принимает условия использования и нажимает «Зарегистрироваться». Система создает аккаунт с ролью user.

2. Загрузка бота. Модератор или администратор открывает экран «Боты», нажимает «Загрузить бота», вводит метаданные, выбирает файл Python-бота и подтверждает загрузку. Система сохраняет бота и активную версию.

3. Поиск ботов. Пользователь открывает экран «Боты», вводит запрос и применяет фильтры по названию, языку, версии, статусу, тегам или владельцу. Система показывает результаты в таблице.

4. Просмотр карточки бота. Пользователь открывает карточку бота и видит свойства, описание, файл, статистику, связанные матчи и настройки запуска. Для файла доступно скачивание.

5. Создание матча. Пользователь выбирает двух ботов, размер поля, условие победы и лимиты запуска. Система создает матч, запускает активные версии ботов и сохраняет результат.

6. Просмотр матча. Пользователь открывает карточку матча, смотрит участников, правила, статус, результат, replay-поле, список ходов и технические сообщения.

7. Просмотр логов. Модератор или администратор открывает список логов, ищет фрагмент текста и фильтрует записи по уровню, боту, матчу и статусу.

8. Статистика. Пользователь открывает экран статистики, задает период, правила или ботов. Система строит агрегаты: число матчей, победы, поражения, ничьи, ошибки, среднюю длительность и среднее число ходов.

9. Конструктор отчета. Пользователь открывает экран «Статистика → Кастомный отчёт», выбирает источник данных, задаёт многокритериальные фильтры, выбирает атрибуты для осей X и Y, тип визуализации и нажимает «Построить». Система строит диаграмму и позволяет сохранить результат отчёта.

10. Настройки приложения. Администратор открывает настройки, меняет лимиты выполнения, лимиты памяти, параметры логирования и роли пользователей. Система валидирует значения и сохраняет их.

5.4. Сценарии импорта данных

Массовая загрузка данных выполняется через экран «Импорт/Экспорт». Пользователь с правами модератора или администратора выбирает вкладку импорта, загружает JSON-файл дампа приложения и подтверждает операцию. Система проверяет структуру файла, восстанавливает пользователей, ботов, версии, статистику, матчи, события, отчеты, настройки и историю операций. Результат импорта записывается в `import_export_history`.

Ручная загрузка данных выполняется через специализированные формы. Бот добавляется на экране «Загрузить бота»: пользователь вводит название, версию, теги, описание и загружает Python-файл. Матч создается на экране «Создать матч»: пользователь выбирает ботов и параметры запуска. Настройки и роли пользователей меняются на экране «Настройки».

5.5. Представление данных

Данные отображаются в виде карточек, таблиц, графиков и replay-поля. Список ботов и список матчей представлены таблицами с поиском и фильтрами. Карточка бота показывает свойства решения, статистику и связанные матчи. Карточка матча показывает метаданные, историю статусов, ходы и логи. Для просмотра состояния партии пользователь использует элементы управления replay: play/pause, шаг вперед/назад и ползунок номера хода.

Поиск реализован на отдельных экранах и через общий поиск. Пользователь может искать ботов по идентификатору, названию, языку, версии, статусу, тегам и владельцу, а логи - по тексту сообщения, уровню, источнику, боту и матчу.

5.6. Анализ данных

Анализ данных выполняется на экранах «Статистика» и «Кастомный отчёт». На экране «Статистика» пользователь видит обзорные агрегаты и диаграммы по ботам и матчам. На экране «Кастомный отчёт» пользователь выбирает источник данных, многокритериальные фильтры, атрибуты для осей X и Y и тип визуализации. После

построения система отображает диаграмму и позволяет сохранить результат отчёта. Результат можно сохранить в системе и повторно открыть из списка сохранённых отчётов

5.7. Экспорт данных

Экспорт выполняется через экран «Импорт/Экспорт». Пользователь с правами модератора или администратора нажимает кнопку экспорта, после чего система формирует полный JSON-дамп всех данных приложения. Отдельный экспорт отдельных сущностей не используется.

5.8. Вывод о преобладающих операциях

Для данного решения операции записи и чтения используются примерно в равной степени, но в разных частях системы. При выполнении матча преобладает запись: создается карточка запуска, сохраняются ходы, логи, статусы и агрегаты. При работе пользователя с интерфейсом преобладает чтение: просмотр списков, карточек, replay, логов и статистики. Для основной пользовательской нагрузки важна оптимизация чтения последних матчей и истории событий, а для фонового выполнения - эффективная пакетная запись событий.

6. МОДЕЛЬ ДАННЫХ

6.1. Нереляционная модель MongoDB

Для нереляционной модели используется MongoDB. Основная идея модели состоит в том, что бот, его версия, статистика, матч и поток событий матча хранятся в естественном для предметной области виде. Один матч представлен карточкой запуска и отдельным потоком событий, включающим ходы и технические сообщения. Такая модель удобна для платформы бесконечных крестиков-ноликов, потому что структура событий матча полуструктурированная, а аналитика может считаться через aggregation pipeline.

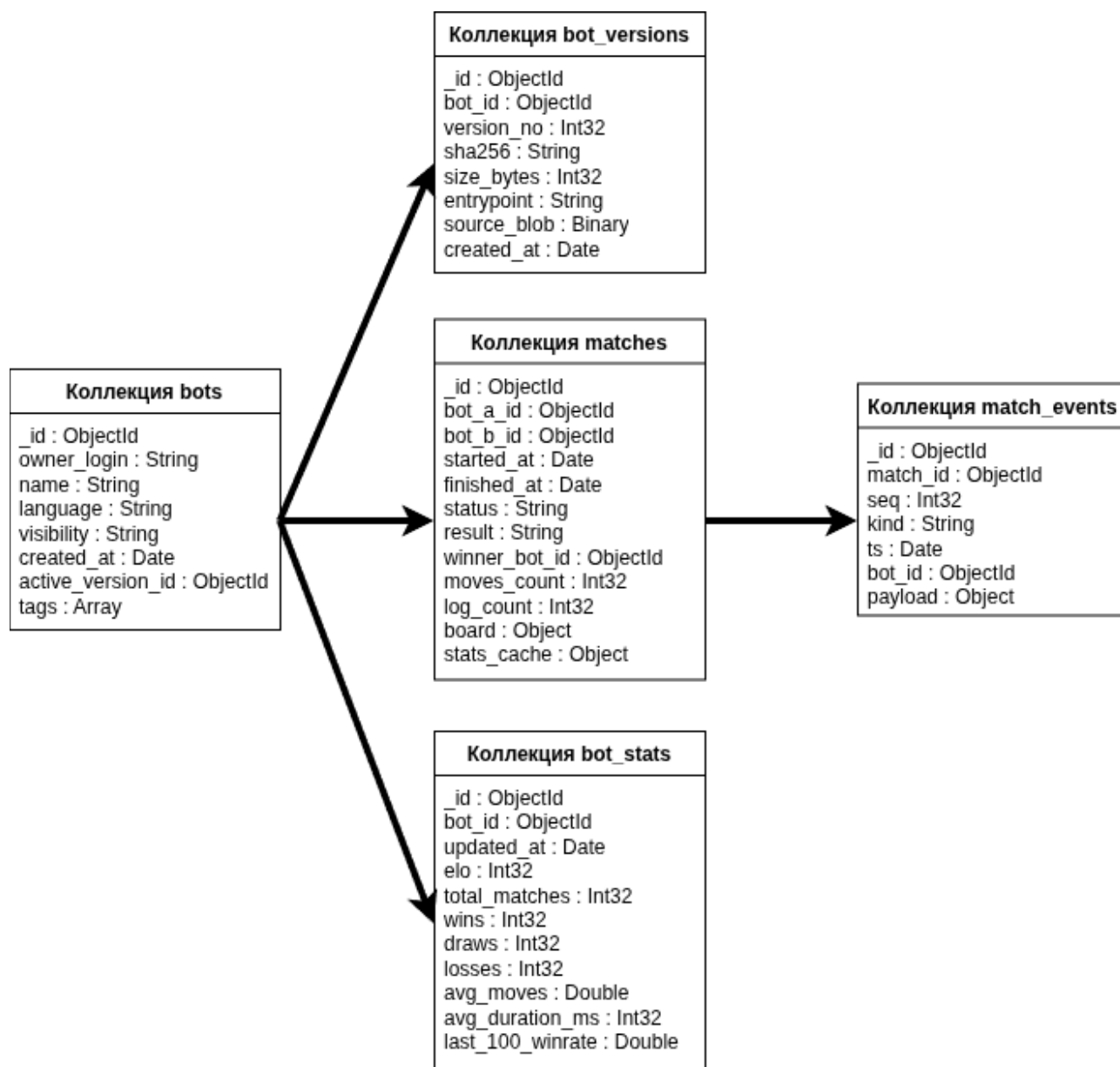


Рисунок 3. Графическое представление нереляционной модели данных.

Коллекция	Назначение
users	Пользователи системы, email, пароль/хеш, роль и служебные признаки.
bots	Карточки пользовательских ботов: владелец, имя, язык, видимость, теги, активная версия.
bot_versions	Сохраненные версии решений: номер версии, контрольная сумма, размер, файл и точка входа.
bot_stats	Агрегированная статистика по ботам:

	рейтинг, число матчей, победы, ничьи, поражения, средние значения.
matches	Карточки запусков матчей: участники, время, статус, результат, параметры поля и кэш статистики.
match_events	Поток событий матча: ходы и логи с порядковым номером, временем, ботом-источником и payload.
reports	Сохраненные конфигурации пользовательских отчетов.
import_export_history	История операций импорта и экспорта.
app_settings	Параметры приложения: лимиты выполнения, памяти, логирования.
log_notes	Заметки и правки к логам.

В качестве основных BSON-типов используются ObjectId - 12 байт, Date - 8 байт, Int32 - 4 байта, Double - 8 байт, String - строка переменной длины, Binary - бинарные данные переменной длины.

Для расчетов принимаются средние размеры объектов: документ bots - 214 байт, bot_versions - 1237 байт, bot_stats - 189 байт, matches - 353 байта, событие-ход в match_events - 143 байта, событие-лог в match_events - 190 байт. Размер BSON-документа складывается из полезных данных полей и служебной информации BSON: длины документа, байтов типов, имен полей, завершающих нулей, длины строк и оболочек вложенных документов.

Для дальнейших расчетов принимаются допущения: в системе хранится 100 ботов, для каждого бота хранится одна активная версия, средний размер архива решения - 1024 байта, один матч содержит в среднем 225 ходов и 35 лог-сообщений.

Оценка общего объема данных при количестве матчей n :

$$V_{\text{nosql}}(n) = 100 \cdot (214 + 1237 + 189) + n \cdot (353 + 225 \cdot 143 + 35 \cdot 190)$$

$$V_{\text{nosql}}(n) = 164000 + 39178n \text{ байт}$$

Направление роста модели линейное: $V_{\text{nosql}}(n) = O(n)$. Основной рост дает коллекция match_events, так как она хранит поток ходов и логов каждого матча.

6.2. Примеры документов и запросов MongoDB

Пример документа из коллекции bots:

```
{
```

```

    "_id": ObjectId("67f000000000000000000001"),
    "owner_login": "student01",
    "name": "DiagonalHunter",
    "language": "python",
    "visibility": "public",
    "created_at": ISODate("2026-03-01T12:00:00Z"),
    "active_version_id": ObjectId("67f0000000000000000000010"),
    "tags": ["aggressive", "baseline"]
  }

```

Пример события-хода в коллекции match_events:

```

{
  "match_id": ObjectId("67f00000000000000000000100"),
  "seq": 17,
  "kind": "move",
  "ts": ISODate("2026-03-20T15:00:02.511Z"),
  "bot_id": ObjectId("67f00000000000000000000001"),
  "payload": { "mark": "X", "x": 12, "y": -4, "decision_ms": 17 }
}

```

Публикация новой версии бота выполняется двумя запросами: вставкой новой версии и обновлением активной версии в карточке бота.

```

db.bot_versions.insertOne({ bot_id: ObjectId("...0001"), version_no: 4, sha256: "bbbb...", size_bytes: 1024,
  entryptpoint: "main.py", source_b
db.bots.updateOne({ _id: ObjectId("...0001") }, { $set: { active_version_id: ObjectId("...0011") } })

```

Получение истории событий матча выполняется чтением карточки матча и постраничным чтением событий, отсортированных по полю seq.

```

db.matches.findOne({ _id: ObjectId("67f00000000000000000000100") })
db.match_events.find({ match_id: ObjectId("67f00000000000000000000100") })
.sort({ seq: 1 }).skip(0).limit(100)

```

Пересчет статистики бота по завершенным матчам может быть выполнен средствами aggregation pipeline.

```

db.matches.aggregate([
  { $match: { $or: [{ bot_a_id: ObjectId("...0001") }, { bot_b_id: ObjectId("...0001") }], status: "finished" } },
  { $group: { _id: null, total_matches: { $sum: 1 }, wins: { $sum: { $cond: [{ $eq: ["$winner_bot_id",
    ObjectId("...0001")] }, 1, 0] } } } }
])

```

Сценарий	Число запросов	Коллекции
Публикация версии бота	2	bot_versions, bots
Сохранение результата матча	$3 + \text{ceil}((E + L) / \beta)$	matches, match_events, bot_stats

Просмотр матча	$1 + \text{ceil}((E + L) / p)$	matches, match_events
Просмотр карточки бота и статистики	$2 + \text{ceil}(r / p_r)$	bots, bot_stats, matches

В формулах E - число ходов, L - число логов, β - размер батча для insertMany, p - размер страницы событий, r - число выводимых матчей, p_r - размер страницы списка матчей.

6.3. Аналог модели для SQL СУБД

В реляционной модели используется PostgreSQL как аналог SQL СУБД. Бот, версия, статистика, матч, ходы и логи раскладываются по таблицам с фиксированной структурой. Такая модель обычно занимает меньше памяти, но требует большего числа таблиц и объединений при работе со сложными сценариями.

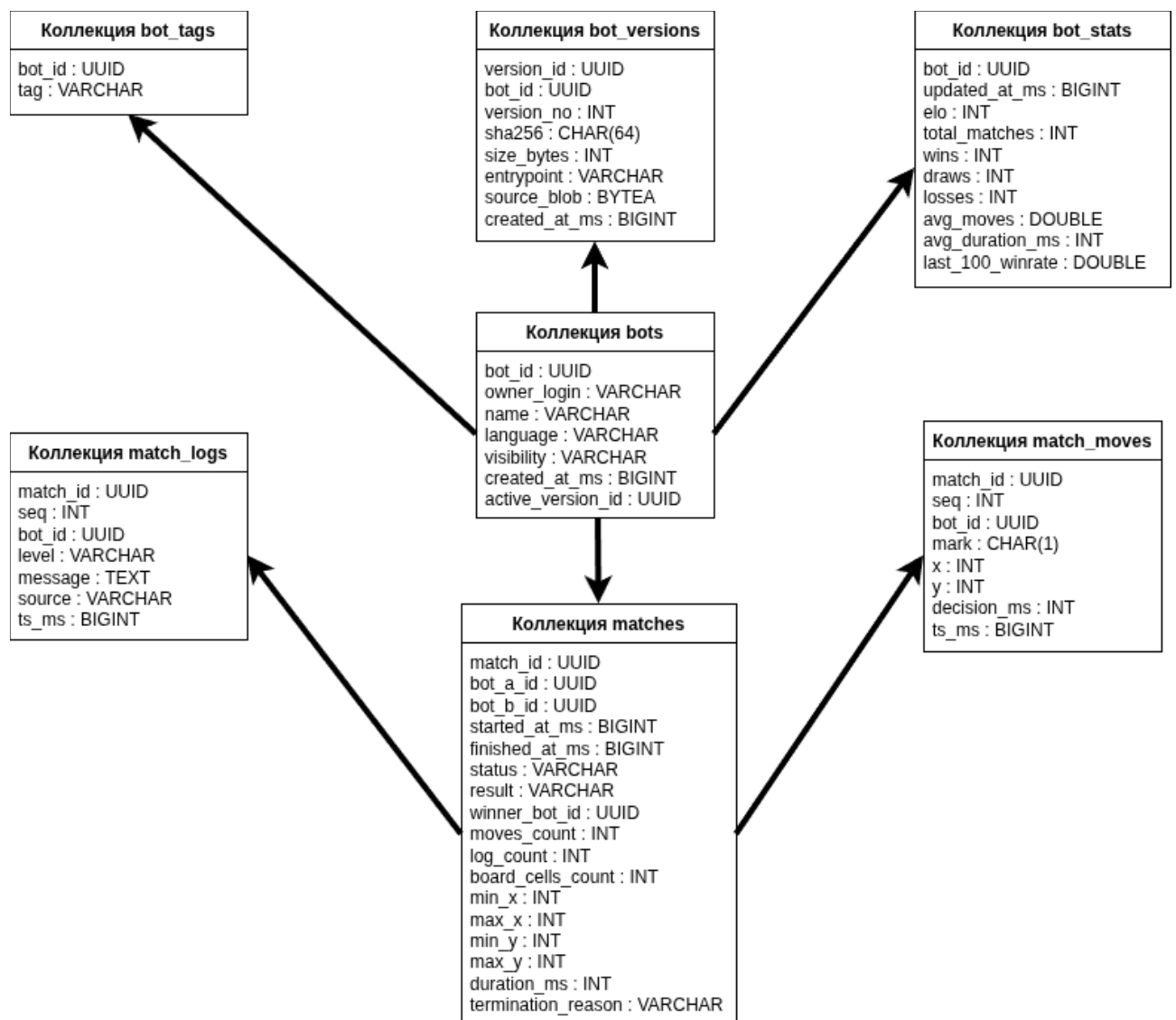


Рисунок 4. Графическое представление реляционной модели данных.

Таблица	Назначение
bots	Карточки ботов.
bot_tags	Теги ботов в нормализованной форме.
bot_versions	Версии решений и файлы.
bot_stats	Агрегированная статистика.
matches	Карточки матчей.
match_moves	История ходов.
match_logs	Технические сообщения и ошибки.

Для расчетов принимаются средние размеры строк: bots - 75 байт, bot_tags - 50 байт на два тега одного бота, bot_versions - 1147 байт, bot_stats - 64 байта, matches - 140 байт, match_moves - 57 байт, match_logs - 96 байт. Оценка выполнена без учета индексов и свободного места в страницах.

Оценка общего объема данных при количестве матчей n:

$$V_{sql}(n) = 100 \cdot (75 + 50 + 1147 + 64) + n \cdot (140 + 225 \cdot 57 + 35 \cdot 96)$$

$$V_{sql}(n) = 133600 + 16325n \text{ байт}$$

Пример SQL-запроса публикации новой версии бота:

```
INSERT INTO bot_versions(version_id, bot_id, version_no, sha256, size_bytes, entryptpoint, source_blob,
created_at_ms)
VALUES ('67f00000-0000-0000-0000-000000000011', '67f00000-0000-0000-0000-000000000001', 4, 'bbbb...',
1024, 'main.py', E'\xBEEF', 1775000000
UPDATE bots SET active_version_id = '67f00000-0000-0000-0000-000000000011'
WHERE bot_id = '67f00000-0000-0000-0000-000000000001';
```

Получение истории матча в SQL-модели требует чтения карточки матча и отдельных таблиц ходов и логов.

```
SELECT * FROM matches WHERE match_id = '67f00000-0000-0000-0000-0000000000100';
SELECT * FROM match_moves WHERE match_id = '67f00000-0000-0000-0000-0000000000100' ORDER BY seq LIMIT 100 OFFSET 0;
SELECT * FROM match_logs WHERE match_id = '67f00000-0000-0000-0000-0000000000100' ORDER BY seq LIMIT 100 OFFSET 0;
```

6.4. Сравнение моделей

Критерий	MongoDB	SQL-модель
Объем хранения	$V_{nosql}(n) = 164000 + 39178n$ байт	$V_{sql}(n) = 133600 + 16325n$ байт

Сохранение матча	$3 + \text{ceil}((E + L) / \beta)$, 3 коллекции	$3 + \text{ceil}(E / \beta m) + \text{ceil}(L / \beta l)$, 4 таблицы
Просмотр матча	$1 + \text{ceil}((E + L) / p)$, 2 коллекции	$1 + \text{ceil}(E / pm) + \text{ceil}(L / pl)$, 3 таблицы
Расширение событий	Добавление новых полей payload без изменения схемы	Необходимы новые столбцы или таблицы
Естественность модели	Высокая: матч и поток событий соответствуют предметной области	Средняя: события разделяются на таблицы ходов и логов
Компактность	Ниже из-за BSON-оболочки и повторения имен полей	Выше при прочих равных

По удельному объему хранения SQL-модель компактнее, так как не повторяет имена полей в каждом документе и хранит события в более узких строках. Однако MongoDB лучше соответствует основному сценарию проекта: матч сохраняется как карточка, а вся история представлена единым потоком событий. Это уменьшает сложность разработки replay, логов и расширяемой аналитики.

Для проекта «Платформа для запуска пользовательских программ-ботов для бесконечных крестиков-ноликов» нереляционная модель MongoDB является более подходящей основной моделью хранения. Реляционная модель остается рабочей альтернативой, но для данной предметной области она менее естественна и требует большего числа структур хранения при работе с матчами и событиями.

7. РАЗРАБОТАННОЕ ПРИЛОЖЕНИЕ

7.1. Архитектура и модули

Разработанное приложение состоит из четырех основных контейнеров: db, backend, bot-runner и frontend. Контейнер db использует образ mongo:7 и хранит данные в volume mongo_data. Backend реализован на FastAPI и предоставляет REST API. Bot-runner запускает пользовательский код в ограниченной среде. Frontend реализован на React + Vite и раздается через nginx.

Компонент	Технологии	Назначение
frontend	React 18, Vite, TypeScript, MUI, Radix UI, Recharts	Интерфейс пользователя, таблицы, формы, графики, маршрутизация.

backend	Python, FastAPI, Pydantic, pymongo, uvicorn	REST API, работа с MongoDB, авторизация, импорт/экспорт, статистика.
bot-runner	Python, FastAPI, изолированный контейнер	Запуск пользовательских Python-ботов и возврат ходов.
db	MongoDB 7	Хранение пользователей, ботов, версий, матчей, событий, отчетов и настроек.
docker-compose	Docker Compose	Сборка и запуск всех сервисов единой командой.

7.2. API приложения

Backend предоставляет набор маршрутов с префиксом `/api`. Основные группы маршрутов перечислены в таблице.

Для табличных представлений ботов, матчей, логов и пользователей реализована server-side пагинация. Клиент передаёт параметры `page` и `page_size`, а backend возвращает элементы текущей страницы, общее количество записей и количество страниц.

Группа	Маршруты	Назначение
auth	POST <code>/login</code> , POST <code>/register</code> , POST <code>/forgot-password</code>	Авторизация и регистрация.
bots	GET <code>/bots</code> , POST <code>/bots</code> , GET/PUT <code>/bots/{id}</code> , GET <code>/download</code>	Работа с ботами и файлами.
matches	GET <code>/matches</code> , POST <code>/matches</code> , GET/PUT <code>/matches/{id}</code>	Создание и просмотр матчей.
logs	GET <code>/logs</code> , GET/PUT <code>/logs/{id}</code> , GET <code>/download</code>	Просмотр, поиск и выгрузка логов.
statistics	GET <code>/statistics</code>	Получение статистических агрегатов.
reports	POST <code>/preview</code> , POST <code>/reports</code> , GET <code>/reports</code> POST	Построение предпросмотра кастомного отчёта по

	/reports/preview, POST /reports, GET /reports	выбранным X/Y-осям, сохранение и получение сохранённых отчётов.
import_export	GET /history, POST /imports, POST /exports	Импорт, экспорт и история операций.
settings	GET /settings, PUT /settings, maintenance actions	Настройки приложения и служебные операции.
users	GET /users, PATCH /users/{id}/role	Управление пользователями и ролями.
sandbox	GET /health, GET /protocol, POST /run	Проверка и запуск бота в песочнице.

7.3. Используемые технологии

В frontend-части используются React, TypeScript, Vite, React Router, Material UI, Radix UI, Recharts и набор собственных компонентов интерфейса. В backend-части используются FastAPI, Pydantic, pymongo и uvicorn. Для хранения используется MongoDB 7. Для запуска и сборки используются Docker и Docker Compose.

Выбор FastAPI связан с удобством описания REST API и моделей данных. Выбор React + Vite обеспечивает быструю разработку интерфейса. MongoDB выбрана как основная СУБД, поскольку структура матчей, логов и отчетов является гибкой и полуструктурированной.

7.4. Снимки экрана приложения

В репозитории присутствует общий макет интерфейса ui_mockup.png. Он показывает основные экраны приложения: вход, регистрацию, обзор, список ботов, создание и просмотр матчей, статистику, отчеты, импорт/экспорт, логи и настройки. На рисунке 5 приведен общий макет экранов.

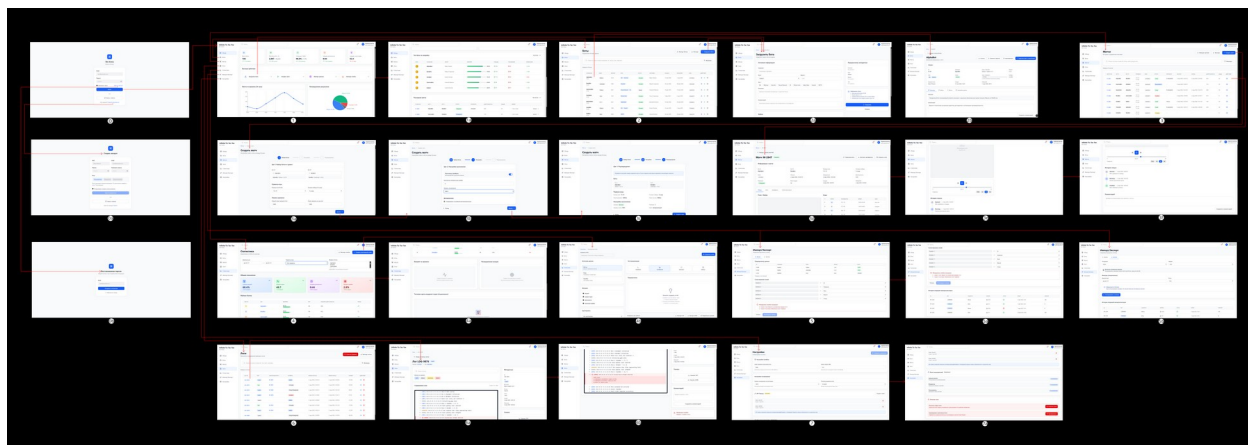


Рисунок 5. Общий макет интерфейса из репозитория проекта.

7.5. Текущее состояние реализации

В текущем состоянии проекта реализованы основные страницы интерфейса, backend-маршруты, демонстрационное наполнение базы, запуск контейнеров, роли пользователей, работа с ботами, матчами, логами, статистикой, отчетами, импортом и экспортом. В репозитории также добавлены два примера ботов для ручной загрузки: `examples/bots/smart_line_bot.py` и `examples/bots/aggressive_line_bot.py`.

Для проверки приложения предусмотрены демонстрационные пользователи: `admin@arena.local / admin123`, `moderator@arena.local / moderator123` и `user@arena.local / user123`. Новый пользователь при регистрации получает роль `user`, а администратор может изменить роль в разделе настроек.

8. ВЫВОД

В ходе выполнения индивидуального домашнего задания разработан прототип веб-приложения Bot Arena Prototype для хранения, запуска и анализа пользовательских ботов игры «бесконечные крестики-нолики». Реализованы основные пользовательские сценарии: регистрация и вход, загрузка ботов, поиск и просмотр карточек, создание матчей, просмотр replay и логов, статистика, конструктор отчетов, импорт, экспорт и настройки приложения.

В работе построена нереляционная модель MongoDB и SQL-аналог на PostgreSQL. Сравнение показало, что SQL-модель компактнее по удельному объему хранения, но MongoDB-модель лучше соответствует предметной области и проще расширяется при появлении новых типов событий, логов и метрик. Поэтому MongoDB является более подходящей основной СУБД для данного решения.

К недостаткам текущей версии можно отнести учебный характер авторизации, ограниченный набор проверок загружаемых ботов, отсутствие промышленной очереди задач и неполную автоматизацию обновления статистики в реальном времени. В будущем решение можно развивать за счет добавления JWT/refresh-токенов, очереди фоновых задач, расширенного sandbox, индексов MongoDB, CI-тестов, WebSocket-обновлений и более подробной системы прав доступа.

ПРИЛОЖЕНИЕ А. ДОКУМЕНТАЦИЯ ПО СБОРКЕ И РАЗВЕРТЫВАНИЮ

Для запуска проекта требуется установленный Docker и Docker Compose. Все сервисы описаны в файле `docker-compose.yml`.

Порядок запуска:

```
git clone https://github.com/moevm/nsql1h26-xo
cd nsql1h26-xo
docker compose build --no-cache
docker compose up
```

После запуска интерфейс доступен по адресу <http://127.0.0.1:3000>, а API - по адресу <http://127.0.0.1:8000/api/health>. Если приложение запущено корректно, health endpoint backend возвращает {"status": "ok"}.

ПРИЛОЖЕНИЕ Б. ИНСТРУКЦИЯ ПОЛЬЗОВАТЕЛЯ

Для входа в систему можно использовать демонстрационные учетные записи: администратор `admin@arena.local` / `admin123`, модератор `moderator@arena.local` / `moderator123`, пользователь `user@arena.local` / `user123`.

Чтобы загрузить бота, необходимо войти под модератором или администратором, открыть экран «Боты», нажать «Загрузить бота», заполнить название, версию, теги, описание и выбрать Python-файл. После загрузки бот появится в списке и сможет участвовать в матчах.

Чтобы создать матч, необходимо открыть экран «Матчи», нажать «Создать матч», выбрать двух ботов, задать параметры поля и лимиты запуска, затем подтвердить создание. После завершения матча можно открыть его карточку, посмотреть результат, replay-поле, список ходов и логи.

Чтобы выполнить импорт или экспорт, необходимо открыть экран «Импорт/Экспорт». Для импорта выбирается JSON-файл дампа приложения. Для экспорта нажимается кнопка формирования дампа, после чего пользователь получает машиночитаемый JSON-файл.

Чтобы построить статистику, необходимо открыть экран «Статистика» или «Конструктор отчета», выбрать фильтры и метрики. Система покажет агрегированные показатели и визуализацию.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Официальная документация MongoDB. URL: <https://www.mongodb.com/docs/> (дата обращения: 26.05.2026).
2. Официальная документация FastAPI. URL: <https://fastapi.tiangolo.com/> (дата обращения: 26.05.2026).
3. Официальная документация React. URL: <https://react.dev/> (дата обращения: 26.05.2026).

4. Официальная документация Docker Compose. URL: <https://docs.docker.com/compose/> (дата обращения: 26.05.2026).
5. Официальная документация PostgreSQL. URL: <https://www.postgresql.org/docs/> (дата обращения: 26.05.2026).